Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №5

з дисципліни
«Алгоритми і структури даних»

Виконав:

студент групи ІМ-43
Калєнченко Михайло Олександрович
номер у списку групи: 12

Перевірила:

Молчанова А.А.

Київ 2024

Завдання:

1. Написати програму розв’язання задачі пошуку (за варіантом) у двовимірному масиві (матриці) методом двійкового пошуку. Алгоритм двійкового пошуку задається варіантом завдання.

2. Розміри матриці m та n взяти самостійно у межах від 7 до 10.

3. При тестуванні програми необхідно підбирати такі вхідні набори початкових значень матриці, щоб можна було легко відстежити коректність виконання пошуку і ця коректність була б протестована для всіх можливих випадків. З метою тестування дозволяється використовувати матриці меншого розміру.

Варіант № 12

Задано квадратну матрицю дійсних чисел $\boldsymbol{A}[\boldsymbol{n},\boldsymbol{n}]$. Визначити присутність у побічній діагоналі матриці заданого дійсного числа $\boldsymbol{X}$ і його місцезнаходження (координати) методом двійкового пошуку (Алгоритм №2), якщо елементи цієї діагоналі впорядковані за незменшенням.

Текст програми:

```
#include <stdio.h>
#include <windows.h>

int find_X (int secondary_diagonal_of_matrix[], int n, int X) {
  int L = 0;
  int R = n - 1;
  while (L <= R) {
     int mid = ((L + R) / 2);
     if ((secondary_diagonal_of_matrix[mid] == X) && (L == R)) {
        return mid;
     }
     else if (secondary_diagonal_of_matrix[mid] < X) {
        L = mid + 1;
     }
     else {
        R = mid;
     }
  }
  return -1;
}

int main(void) {
  int n = 7;
  int secondary_diagonal_of_matrix[7];
  int matrix [7][7] = {
     {0, 0, 0, 0, 0, 0, 1},
     {0, 0, 0, 0, 0, 1, 0},
     {0, 0, 0, 0, 1, 0, 0},
     {0, 0, 0, 2, 0, 0, 0},
     {0, 0, 2, 0, 0, 0, 0},
     {0, 2, 0, 0, 0, 0, 0},
     {3, 0, 0, 0, 0, 0, 0},
  };
  printf("Matrix is:\n");
  for (int i = 0; i < 7; i++) {
     for (int j = 0; j < 7; j++) {
        printf("%d ", matrix[i][j]);
     }
     printf("\n");
  }
  printf("\n----------------------------\n\n");
  for (int i = 0; i < n; i++) {
     secondary_diagonal_of_matrix[i] = matrix[i][n-i-1];
  }
  int X;
  printf("Enter X:");
```

```
  scanf("%d", &X);
  printf("----------------------------\n");
  int result = find_X(secondary_diagonal_of_matrix, n, X);
  if (result != -1) {
     printf("Secondary diagonal is: ");
     for (int i = 0; i < n; i++) {
        secondary_diagonal_of_matrix[i];
        printf("%d ",secondary_diagonal_of_matrix[i]);
     }
     int position = n - result - 1;
     printf("\nThe X was found on [%d][%d] position",  result,
position);
  }
  else {
     printf("The X was not found! :(");
  }
  Sleep(10000);
  return 0;
}
```

Вхідні дані:

```
{0, 0, 0, 0, 0, 0, 1},
{0, 0, 0, 0, 0, 1, 0},
{0, 0, 0, 0, 1, 0, 0},
{0, 0, 0, 2, 0, 0, 0},
{0, 0, 2, 0, 0, 0, 0},
{0, 2, 0, 0, 0, 0, 0},
{3, 0, 0, 0, 0, 0, 0},
```

```
{1,1,1,4},
{1,1,4,1},
{1,5,1,1},
{6,1,1,1}
```

```
{1,1,1,1,1,1,1,1,1,5},
{1,1,1,1,1,1,1,1,6,1},
{1,1,1,1,1,1,1,6,1,1},
{1,1,1,1,1,1,6,1,1,1},
{1,1,1,1,1,7,1,1,1,1},
{1,1,1,1,7,1,1,1,1,1},
{1,1,1,7,1,1,1,1,1,1},
{1,1,7,1,1,1,1,1,1,1},
{1,8,1,1,1,1,1,1,1,1},
{8,1,1,1,1,1,1,1,1,1},
```

Скріншоти тестів програми:

```
Matrix is:
1 1 1 4
1 1 4 1
1 5 1 1
6 1 1 1

----------------------------

Enter X:4
----------------------------
Secondary diagonal is: 4 4 5 6
The X was found on [0][3] position
```

```
Matrix is:
0 0 0 0 0 0 1
0 0 0 0 0 1 0
0 0 0 0 1 0 0
0 0 0 2 0 0 0
0 0 2 0 0 0 0
0 2 0 0 0 0 0
3 0 0 0 0 0 0

----------------------------

Enter X:2
----------------------------
Secondary diagonal is: 1 1 1 2 2 2 3
The X was found on [3][3] position
```

```
Matrix is:
0 0 0 0 0 0 1
0 0 0 0 0 1 0
0 0 0 0 1 0 0
0 0 0 2 0 0 0
0 0 2 0 0 0 0
0 2 0 0 0 0 0
3 0 0 0 0 0 0

----------------------------

Enter X:1
----------------------------
Secondary diagonal is: 1 1 1 2 2 2 3
The X was found on [0][6] position
```

```
Matrix is:
1 1 1 1 1 1 1 1 1 5
1 1 1 1 1 1 1 1 6 1
1 1 1 1 1 1 1 6 1 1
1 1 1 1 1 1 6 1 1 1
1 1 1 1 1 7 1 1 1 1
1 1 1 1 7 1 1 1 1 1
1 1 1 7 1 1 1 1 1 1
1 1 7 1 1 1 1 1 1 1
1 8 1 1 1 1 1 1 1 1
8 1 1 1 1 1 1 1 1 1

----------------------------

Enter X:6
----------------------------
Secondary diagonal is: 5 6 6 6 7 7 7 7 8 8
The X was found on [1][8] position
```

```
Matrix is:
1 1 1 1 1 1 1 1 1 5
1 1 1 1 1 1 1 1 6 1
1 1 1 1 1 1 1 6 1 1
1 1 1 1 1 1 6 1 1 1
1 1 1 1 1 7 1 1 1 1
1 1 1 1 7 1 1 1 1 1
1 1 1 7 1 1 1 1 1 1
1 1 7 1 1 1 1 1 1 1
1 8 1 1 1 1 1 1 1 1
8 1 1 1 1 1 1 1 1 1

----------------------------

Enter X:7
----------------------------
Secondary diagonal is: 5 6 6 6 7 7 7 7 8 8
The X was found on [4][5] position
```

Висновок:

У ході виконання лабораторної роботи №2.1 «Алгоритми двійкового пошуку» я засвоїв теоретичні знання та набув навичок у застосуванні методів двійкового пошуку.

Під час виконання завдання я використовував матриці різних розмірів (4-10), що дало мені змогу протестувати програму із різними вхідними даними. Щодо алгоритму пошуку, я набув навичок використання Алгоритму №2,що шукає найлівіший із шуканих елементів, а також теоретично опанував Алгоритм №1, який шукає будь-який із шуканих елементів.

Таким чином, поставлені задачі були успішно вирішені, а алгоритм двійкового пошуку продемонстрував свою ефективність для знаходження елементів у двовимірних масивах, що підтвердило доцільність його застосування для задач пошуку в структурованих даних.